

PROJECT DOCUMENTATION

dvote Loyalty Platform

Spend-based loyalty points and vendor rewards for coffee shops, collected and redeemed with one-time QR codes. Customer app, staff app, vendor dashboard and admin dashboard.

Version 2.0 · 3 October 2026 · replaces v1.2 (NFC, "buy N get 1 free")

Contents

1. Introduction

1.1 Overview · 1.2 Actors and roles · 1.3 Core concepts and business rules

2. Diagrams

2.1 Collecting points · 2.2 Redeeming a reward · 2.3 Entity relationship diagram

3. Flows and Example

3.1 Collect flow · 3.2 Redeem flow · 3.3 What the customer sees · 3.4 Worked example: Joy Corner

4. Database

4.1 Table overview · 4.2 Table definitions · 4.3 Constraints and indexes

5. Technical Architecture

5.1 Tech stack · 5.2 Supporting services · 5.3 Backend decisions · 5.4 QR code security · 5.5 Build order

6. API Endpoints

6.1 Customer app · 6.2 Staff app · 6.3 Vendor dashboard · 6.4 Admin dashboard · 6.5 Error codes

7. Apps and Dashboards

7.1 Staff app · 7.2 Vendor vs admin dashboard

8. Edge Cases and Fraud

8.1 Edge cases · 8.2 Automatic fraud flags

9. Decisions and Roadmap

9.1 Decisions · 9.2 Later versions

What changed in v2.0. The business moved from "buy N coffees, get 1 free" with NFC cards to **spend-based points**: each vendor sets how much money earns a point (e.g. 10 EGP = 1 point) and publishes a **catalogue of rewards** priced in points (e.g. free coffee 300 pts). Points are collected and redeemed with **one-time QR codes** shown in the customer app and scanned by vendor staff in a new **staff app**. Removed: `nfc_tags`, NTAG 424 security, the 10-minute cooldown, `programs`, `wallets`, `scan_sessions`. New: `point_rules`, `rewards`, `cards`, `qr_codes`. Login moved to Supabase Auth (built in v1.2 work). Items new in this version are marked **NEW**; endpoints already implemented are marked **BUILT**.

Introduction

What the platform is, who uses it and the rules it follows.

1.1 Overview

A multi-vendor loyalty platform for coffee shops. Customers earn **points** on every purchase, in proportion to what they spend, and spend those points on **rewards** that each vendor defines. Points belong to one vendor: points earned at Ecuador can only be spent at Ecuador.

Part	Users	Purpose
Customer app (iOS + Android)	Customers	Sign in, see their cards (points per vendor), browse vendors and rewards, show a QR to collect points or redeem a reward
Staff app (iOS + Android) NEW	Vendor staff, branch managers, vendor admins	Scan the customer's QR, enter the receipt total to award points, confirm reward redemptions
Vendor dashboard (web)	Vendor admins, branch managers	Branches, staff, point rule, reward catalogue, reports
Admin dashboard (web)	Platform team	All vendors, onboarding, customer support, point corrections, fraud review

Rewards are paid for by the vendor. For the vendor, the platform is a customer-retention and advertising channel.

v1 scope: collect points by QR, redeem rewards by QR, per-vendor cards, the staff app and both dashboards. Point expiry and push notifications are deferred (see Roadmap).

1.2 Actors and roles

Role	Scope	Can do
Customer	Own account	Sign in with Google or Facebook, see cards and history, show collect / redeem QR codes
Staff	One branch	In the staff app: scan QR codes, enter receipt totals, confirm redemptions, see their own activity today
Branch manager	One branch	Everything staff can do, plus manage the staff of their branch and see branch reports
Vendor admin	One vendor (all branches)	Manage branches, staff, the point rule and the reward catalogue; reports for their vendor; can also scan (choosing the branch)
Platform admin	Everything	Onboard vendors, view all vendors, support customers, correct points, review fraud

Customers use the customer app. Staff, branch managers and vendor admins use the staff app to scan and the vendor dashboard to manage. Platform admins use the admin dashboard.

1.3 Core concepts and business rules

Vendor vs branch. A vendor is the brand (e.g. Ecuador, Joy Corner); a branch is one physical shop. A chain with 6 shops = 1 vendor, 6 branches.

Card. One card per customer per vendor: the customer's points at that vendor. A card is created automatically on the customer's **first purchase** at that vendor; a customer with no points at a vendor has no card for it. Points from any branch of the same vendor add to the same card and never move between vendors.

Point rule NEW. Each vendor has one active rule (its "settings"): *every spend_amount spent earns points_per_spend points*, e.g. every 10 EGP = 1 point. Optional: a minimum purchase and a maximum number of points per receipt. Points are always rounded **down**: 95 EGP at 10 EGP/point = 9 points. Rules are versioned: a new rule applies to purchases from then on; past points never change.

Rewards NEW. Each vendor publishes a catalogue of rewards priced in points, e.g. free coffee 300 · free cheesecake 500 · coffee + cheesecake 750. Rewards can be edited or archived, never deleted; a redemption keeps a copy of the reward's name and price at that moment.

Collecting. The customer shows a QR; staff scan it and enter the receipt total; **the server** applies the vendor's rule and adds the points. The app never decides points.

Redeeming. The customer picks a reward they can afford and shows a redeem QR; staff scan it; the server checks the balance, deducts the reward's points and records the redemption. The customer chooses when to redeem; nothing is automatic. Leftover points stay on the card ($1000 - 300 = 700$).

One-time QR codes NEW. Every QR holds a random single-use code issued by the server, valid for 5 minutes. A screenshot or photo of an old QR is useless.

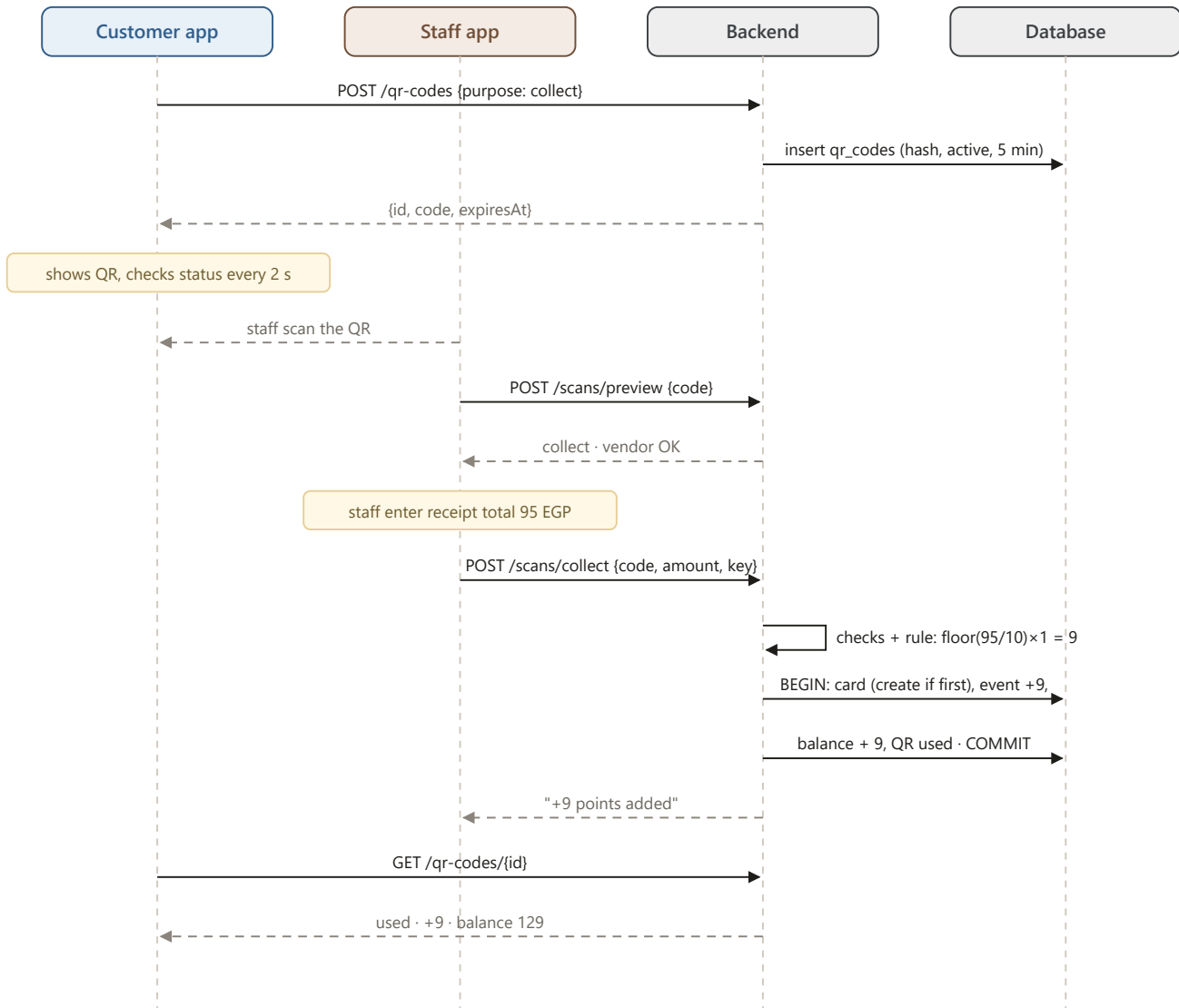
Ledger is the source of truth. A card's balance is a cached total and must always equal the sum of that card's point events (earns positive, redemptions negative, admin adjustments either). Every balance change is written in the same database transaction as its ledger row.

Tenant scoping. Staff only ever see and act on their own vendor (and their own branch, for branch roles).

Diagrams

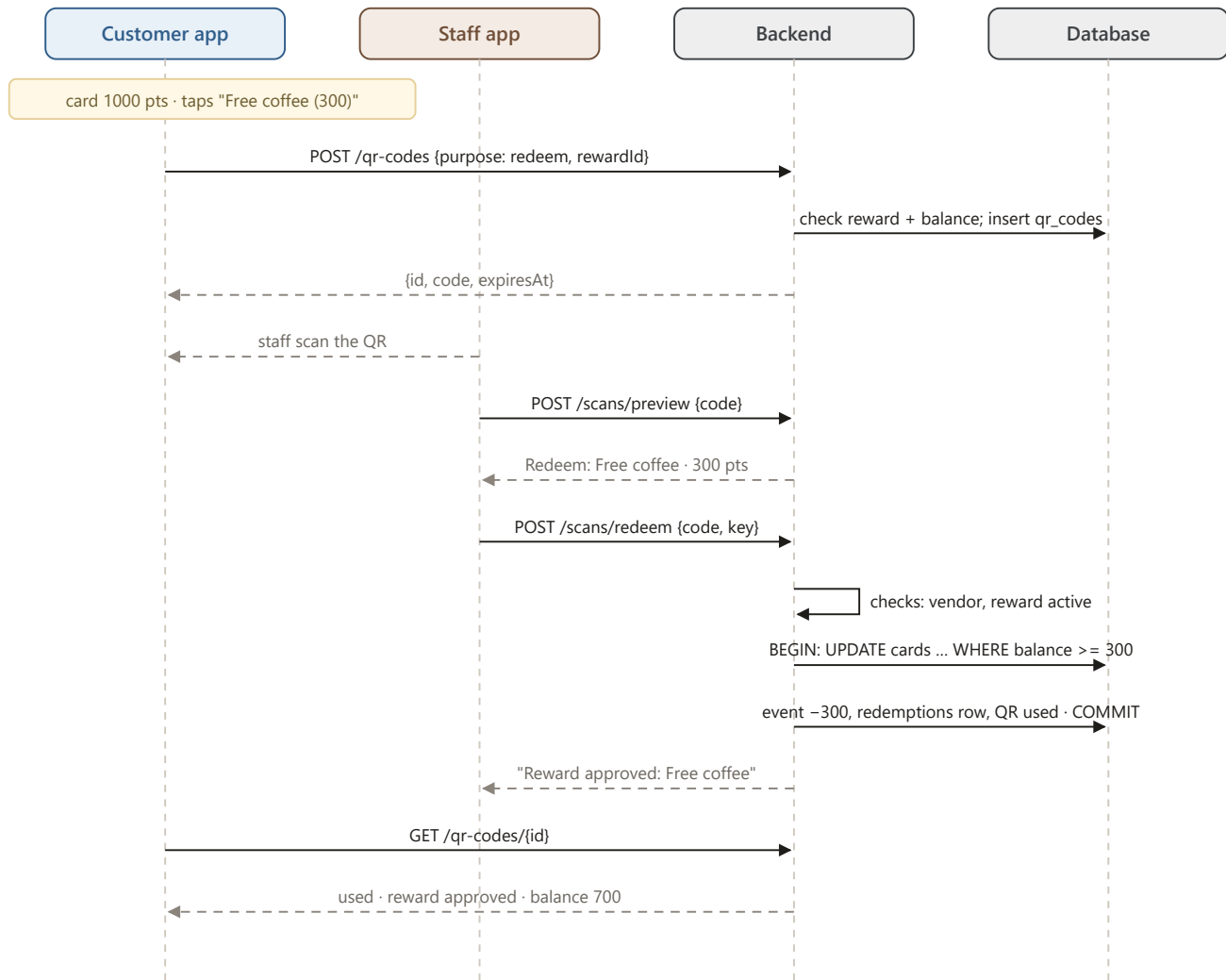
The two main sequence diagrams and the entity relationship diagram. Section 3 explains each flow step by step.

2.1 Sequence diagram: collecting points



Master QR shown (vendor unknown until a staff member scans it). With a vendor QR the only difference is that the QR is tied to one vendor and other vendors' staff are refused.

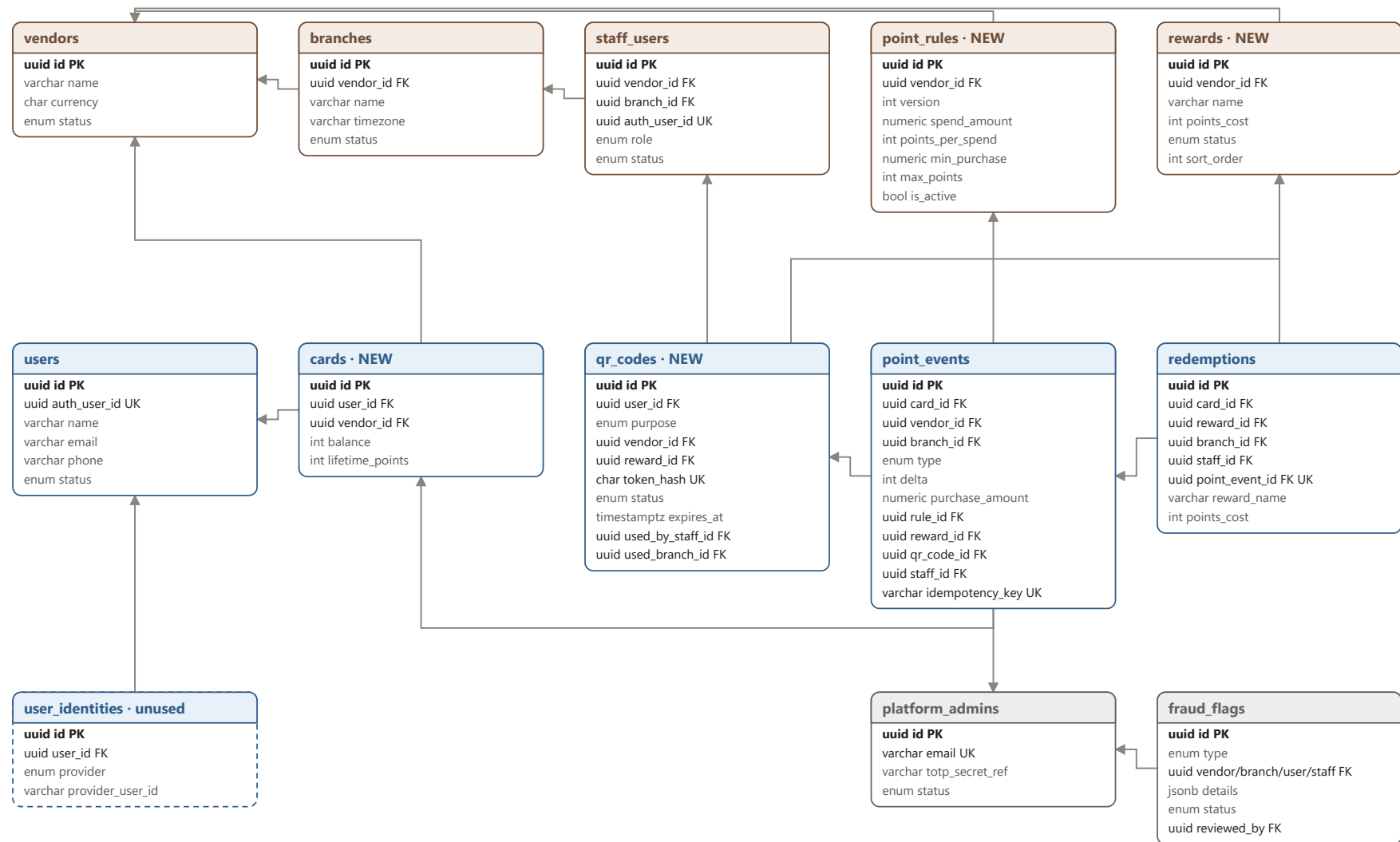
2.2 Sequence diagram: redeeming a reward



Points are deducted only when staff confirm, with a conditional update, so two scans of the same QR or two simultaneous redemptions cannot both succeed.

2.3 Entity relationship diagram

Thirteen tables; `vendors` is the tenant root. Brown = vendor side, blue = customer and activity, grey = platform. Lines point from a foreign key to the table it references; every activity row also carries `vendor_id`, and composite foreign keys guarantee its card, branch, rule and reward all belong to that vendor. Dashed = kept but currently unused.



Flows and Example

How collecting and redeeming work, then a walk-through with a real vendor.

3.1 Collect flow (earning points)

There are two ways to show a collect QR; the server handles both the same way.

QR	Where in the app	Contains	Vendor comes from
Master QR	Vendors list page	One-time code → customer	The staff member who scans it
Vendor QR	A vendor's page	One-time code → customer + that vendor	The QR; other vendors' staff are refused (<code>vendor_mismatch</code>)

In both cases the **branch comes from the scanning staff member**, never from the customer's phone. A vendor admin (who has no branch) picks the branch in the staff app.

1. Customer taps **Collect points** (master or vendor QR). The app calls `POST /api/app/qr-codes` `{purpose:"collect", vendorId?}`; the server stores a `qr_codes` row (status active, expires in 5 min, only the code's hash is stored) and returns the code. Any older active collect QR of that customer is cancelled.
2. The app shows the QR and waits, checking the QR's status every 2 seconds.
3. Staff scan it in the staff app. The app calls `POST /api/vendor/scans/preview` to show what the QR is (collect; vendor allowed).
4. Staff type the receipt total (and optionally the receipt number) and submit `POST /api/vendor/scans/collect` `{code, amount, receiptRef?, branchId?, idempotencyKey}`.
5. The server runs the checks below, computes the points with the vendor's active rule and, in **one transaction**: creates the card if this is the customer's first purchase at this vendor, inserts a `point_events` row (earn, +points, amount, rule version, branch, staff), adds the points to `cards.balance` and `lifetime_points`, and marks the QR used.
6. The staff app shows "**+9 points added**"; the customer app sees the QR is used and shows "+9 points at Joy Corner · balance 129".

Server checks for collect, in order

1. Code exists and is `active` and not expired (`qr_invalid`, `qr_used`, `qr_expired`).
2. Idempotency key already used → return the original result, add nothing.
3. QR is a collect QR (`wrong_qr_type`).
4. Staff active; branch known (`branch_required` for vendor admins); vendor and branch active.
5. Vendor QR: its vendor = the staff member's vendor (`vendor_mismatch`).
6. Customer not blocked (`user_blocked`).
7. Vendor has an active rule (`no_active_rule`); amount valid (`invalid_amount`); points > 0 (`no_points_earned`, e.g. 8 EGP at 10 EGP/point or below the minimum purchase); receipt number not already used at this branch (`duplicate_receipt`).

A failed check returns a clear error to the staff app and changes nothing; the QR stays usable so staff can correct a typo.

3.2 Redeem flow

1. On a vendor's page, the app lists the rewards with their prices and marks the ones the card can afford.
2. Customer taps a reward → `POST /api/app/qr-codes {purpose:"redeem", rewardId}`. The server checks the reward is active and the balance is enough (so the customer gets an early "not enough points"), then issues a redeem QR tied to that vendor and reward.
3. Staff scan it → preview shows "**Redeem: Free coffee · 300 pts**" so staff know what to serve.
4. Staff confirm → `POST /api/vendor/scans/redeem {code, branchId?, idempotencyKey}`.
5. Checks: code valid (as above); it is a redeem QR; its vendor = the staff member's vendor (`vendor_mismatch`); reward still active (`reward_unavailable`); customer not blocked; balance still enough (`insufficient_points`).
6. One transaction:

```
UPDATE cards SET balance = balance - $cost
WHERE id = $card AND balance >= $cost
RETURNING balance;          -- no row ⇒ not enough points ⇒ abort
```

then insert `point_events` (redeem, -cost, reward, branch, staff), insert `redemptions` (with the reward's name and cost copied), mark the QR used.

7. Both apps show "**Reward approved: Free coffee**", branch and time; staff serve the reward.

3.3 What the customer sees

- **My cards:** one card per vendor where the customer has points (none at first). Each card shows the vendor, balance, and progress to the next reward ("120 / 300 to Free coffee").
- **Vendors:** all active vendors with their rule ("10 EGP = 1 point") and the master **Collect points** QR button.
- **Vendor page:** branches, rule, rewards (affordable ones highlighted), the vendor **Collect points** QR and a **Redeem** button per reward.
- **History:** every earn, redemption and correction on a card, with date, branch and amount.

3.4 Worked example: Joy Corner

Joy Corner has a Smouha branch. Its rule: every 10 EGP = 1 point. Rewards: Free coffee 300 · Free cheesecake 500 · Free coffee + cheesecake 750.

Setup (done once)

Table	Row written
vendors	Joy Corner, currency EGP, active
branches	Joy Corner Smouha → Joy Corner
point_rules	version 1, spend_amount 10.00, points_per_spend 1, active
rewards	3 rows: Free coffee 300, Free cheesecake 500, Free coffee + cheesecake 750
staff_users	Sara, role staff, branch Smouha (signs in to the staff app)
users	Karim signs in with Google: one user row (linked to his Supabase account)

Karim's Joy Corner card over time

Visit	What happens	Rows written	Balance	App shows
1	Shows master QR; Sara scans, enters 95 EGP	card created; point_events +9 (rule v1)	9	New card · 9 / 300 to Free coffee
2–20	Buys regularly, total 2,400 EGP	point_events +240 in total	249	249 / 300
21	Spends 8 EGP	nothing (no_points_earned)	249	Sara sees "0 points: below 10 EGP"
22	Vendor QR, spends 520 EGP	point_events +52	301	Free coffee available
22	Redeems Free coffee; Sara scans and confirms	point_events –300; redemptions row	1	"Reward approved: Free coffee"
23	Shows an old screenshot of a QR	nothing (qr_used)	1	Sara sees "This QR was already used"

At any time `cards.balance` equals the sum of Karim's `point_events.delta` for Joy Corner; `lifetime_points` keeps counting up (301) and never decreases.

If Joy Corner changes the rule. Publishing "every 10 EGP = 2 points" adds `point_rules` version 2 and makes it the active one. Purchases from then on earn double; Karim's earlier points stay as they were, and each earn row records which version it used. **If Joy Corner changes a reward** (coffee now 350 points), new redemptions use 350; past redemptions keep the 300 they were made at.

Database

All thirteen tables, their columns, constraints and indexes. Source of truth: `database/dvote_schema.sql`.

4.1 Table overview

#	Table	Side	Holds
1	vendors	Vendor	Brands, e.g. Joy Corner; now with a currency
2	branches	Vendor	Physical shops of a vendor
3	staff_users	Vendor	Vendor admins, branch managers, staff (linked to Supabase Auth)
4	point_rules NEW	Vendor	How spending turns into points, versioned (replaces programs)
5	rewards NEW	Vendor	Each vendor's reward catalogue, priced in points
6	users	Customer	Customers of the app (linked to Supabase Auth)
7	user_identities	Customer	Kept but unused: Supabase Auth tracks Google / Facebook identities
8	cards NEW	Customer	One points balance per customer per vendor (replaces wallets)
9	qr_codes NEW	Activity	Every QR the app shows: one-time, 5 minutes (replaces scan_sessions)
10	point_events	Activity	The ledger: every earn, redemption and correction; append-only
11	redemptions	Activity	Every reward given, with the reward's name and price at that time
12	platform_admins	Platform	Your team (admin dashboard)
13	fraud_flags	Platform	Suspicious activity for admins to review

Removed in v2.0: `nfc_tags`, `programs`, `wallets`, `scan_sessions`. Every table has `id uuid`, `created_at` and `updated_at` (UTC, `updated_at` kept current by a trigger). Business rows are never deleted; they change `status`.

4.2 Table definitions

vendors

Column	Type	Notes
name	varchar(120)	Brand name shown in the app
logo_url, contact_email	varchar, nullable	
currency NEW	char(3), default EGP	Currency of receipt totals and rules
status	enum	active, suspended

branches

Column	Type	Notes
vendor_id	uuid FK → vendors	UNIQUE (id, vendor_id) lets other tables prove a branch belongs to a vendor

Column	Type	Notes
name, address	varchar	e.g. "Smouha"
lat, lng	decimal(9,6), nullable	For the map
timezone	varchar, default Africa/Cairo	For displaying times
status	enum	active, closed

staff_users

Column	Type	Notes
vendor_id	uuid FK → vendors	Always set
branch_id	uuid, nullable	NULL for vendor admins, required for the other roles; must be a branch of the same vendor
name, email	varchar	email unique (case-insensitive)
auth_user_id	uuid, UNIQUE	The Supabase Auth account; passwords live in Supabase
password_hash	varchar, nullable	Legacy, unused
role	enum	vendor_admin, branch_manager, staff
status	enum	active, disabled (disabled = blocked immediately)

point_rules NEW

Points for a purchase = $\text{floor}(\text{amount} \div \text{spend_amount}) \times \text{points_per_spend}$, capped at $\text{max_points_per_purchase}$; 0 if $\text{amount} < \text{min_purchase}$.

Column	Type	Notes
vendor_id	uuid FK → vendors	
version	int ≥ 1	+1 each time the vendor changes its rule
spend_amount	numeric(12,2) > 0	e.g. 10.00 (EGP)
points_per_spend	int > 0, default 1	Points for each spend_amount
min_purchase	numeric(12,2) ≥ 0, default 0	Smaller receipts earn nothing
max_points_per_purchase	int, nullable	Optional cap per receipt
is_active	bool	Only one active rule per vendor
created_by_staff_id	uuid FK → staff_users, nullable	Who published it

rewards NEW

Column	Type	Notes
vendor_id	uuid FK → vendors	
name	varchar(120)	e.g. "Free cheesecake"
description, image_url	varchar, nullable	Shown in the app
points_cost	int > 0	e.g. 500

Column	Type	Notes
status	enum	active, archived (archived rewards disappear from the app)
sort_order	int	Display order

users **(customers)**

Column	Type	Notes
auth_user_id	uuid, UNIQUE	The Supabase Auth account; the row is created on the first API call
name, email, avatar_url	varchar, nullable	From Google / Facebook; editable
phone	varchar, nullable	Optional, not used for login
status	enum	active, blocked

cards **NEW** (replaces wallets)

Column	Type	Notes
user_id, vendor_id	uuid FKs	UNIQUE (user_id, vendor_id): one card per customer per vendor
balance	int ≥ 0	Cached sum of the card's point_events.delta
lifetime_points	int ≥ 0	Total ever earned; never decreases
last_activity_at	timestampz	For sorting cards and reports

qr_codes **NEW** (replaces scan_sessions)

Column	Type	Notes
user_id	uuid FK → users	Whose QR it is
purpose	enum	collect, redeem
vendor_id	uuid FK, nullable	NULL = master QR; set for vendor QRs and always for redeem
reward_id	uuid, nullable	Redeem only; must be a reward of that vendor
token_hash	char(64), UNIQUE	SHA-256 of the code in the QR; the code itself is never stored
status	enum	active, used, expired, cancelled
expires_at	timestampz	Default created_at + 5 minutes
used_at, used_by_staff_id, used_branch_id	nullable	Required once used: who scanned it, where

point_events **(ledger, append-only)**

Column	Type	Notes
card_id, vendor_id	uuid FKs	The card must belong to that vendor

Column	Type	Notes
branch_id	uuid, nullable	Where it happened (NULL only for admin corrections)
type	enum	earn, redeem, adjust
delta	int	+points (earn), -points_cost (redeem), ± (adjust)
purchase_amount NEW	numeric(12,2)	Earn: receipt total
receipt_ref NEW	varchar(64), nullable	Earn: optional receipt number; unique per branch
rule_id NEW	uuid, nullable	Earn: rule version used
reward_id NEW	uuid, nullable	Redeem: reward given
qr_code_id NEW	uuid FK, nullable	The QR scanned; at most one event per QR
staff_id NEW	uuid FK, nullable	Who scanned (earn and redeem)
created_by, reason	nullable	Adjust: platform admin and reason (required)
idempotency_key	varchar, UNIQUE	Sent by the staff app per submit; a retry returns the first result

redemptions

Column	Type	Notes
card_id, vendor_id, reward_id, branch_id	uuid FKs	All must belong to the same vendor
staff_id	uuid FK → staff_users	Who confirmed it
point_event_id	uuid FK, UNIQUE	The matching negative ledger row
reward_name, points_cost	snapshot	Copied from the reward at redemption time

platform_admins, user_identities

Unchanged from v1.2. Platform admins will move to Supabase Auth with two-factor login (adds `auth_user_id`). `user_identities` is kept but not used.

fraud_flags

Column	Type	Notes
type	enum	too_many_collects, large_purchase, branch_spike, staff_spike NEW
vendor_id, branch_id, user_id, staff_id	uuid FKs, nullable	Who it is about (at least one); <code>staff_id</code> replaces <code>tag_id</code>
details	jsonb	The numbers behind the flag
status, reviewed_by	enum, FK	open, dismissed, confirmed; reviewer required once closed

4.3 Constraints and indexes

- One active rule per vendor (partial unique index); `UNIQUE (vendor_id, version)`.

- `cards` : one per (user, vendor); `balance` ≥ 0 .
- `point_events` : earn $\Rightarrow$ delta > 0 with amount, rule, QR, staff, branch; redeem $\Rightarrow$ delta < 0 with reward, QR, staff, branch; adjust $\Rightarrow$ admin and reason. UPDATE and DELETE are blocked by a trigger: corrections are new adjust rows.
- Composite foreign keys on `(..., vendor_id)` : a ledger row's card, branch, rule and reward must all belong to its vendor, so points can never cross vendors even by a bug.
- At most one ledger row per QR code; a receipt number earns once per branch.
- `qr_codes` : redeem $\Rightarrow$ vendor and reward set; collect $\Rightarrow$ no reward; used $\Rightarrow$ used_at, staff and branch set.
- Indexes: `point_events` by (card, time), (branch, time), (vendor, time), (staff, time); rewards by (vendor, status, order); `qr_codes` by (user, time); `fraud_flags` by (status, time).

Technical Architecture

Stack, services, implementation decisions and QR security.

5.1 Tech stack

Layer	Choice	Why for this project
Database	PostgreSQL 18	A money-like ledger needs ACID transactions; partial unique indexes, CHECK constraints, composite keys and triggers enforce the rules in the database itself
ORM	Prisma 7	Generated TypeScript types; hand-written SQL migrations keep the constraints Prisma can't express
Backend	NestJS (TypeScript)	One module per feature, guards for role and tenant scoping
Web dashboard	React + TypeScript (Vite), Ant Design, TanStack Query, Recharts	One app with role-based routes for vendors and platform admins
Customer app NEW	React Native + Expo	Same language as the backend and dashboard; shows QR codes (no NFC needed any more)
Staff app NEW	React Native + Expo (separate app)	Camera QR scanner, receipt-total entry, redemption confirmation
Auth	Supabase Auth	Google / Facebook for customers, email + password for staff; the backend only verifies tokens
API contract	OpenAPI from NestJS	One generated TypeScript client shared by the dashboard and both apps

5.2 Supporting services

- **Supabase Auth:** customers sign in with Google or Facebook; staff are invited by email and set a password. Supabase issues and refreshes tokens; the backend checks them locally with the project's public keys and maps each one to its own `users` or `staff_users` row. Apple sign-in is postponed (paid developer account); an iOS release offering Google/Facebook will likely need it.
- **Email:** Supabase sends staff invites and password resets; production needs its own SMTP provider (the built-in sender is rate-limited).
- **Push notifications** (later): Firebase Cloud Messaging, to confirm points when the app is closed.
- **Hosting:** one backend container plus one managed PostgreSQL. No microservices, queues or Redis in v1.

5.3 Backend decisions

- **Balance changes are atomic.** Earn and redeem run inside `prisma.$transaction` with the ledger insert. Redemption uses a conditional `UPDATE ... WHERE balance >= cost`; marking the QR used is also conditional (`WHERE status = 'active'`), so the same QR can never be processed twice, even by two staff phones at once.
- **Money is exact.** Amounts are `numeric(12,2)` in the database and integer minor units (piastres) in code, never floating point. Points round down.

- **Tenant scoping in one place.** A guard builds the staff member's context (vendor, branch, role) once per request; every vendor-side query filters by it.
- **Idempotency.** The staff app sends a fresh key with each submit; a retry after a lost response returns the original result.
- **Customer confirmation.** While a QR is on screen the customer app checks its status every 2 seconds and shows the result as soon as it is used.
- **Fraud checks:** a scheduled job writes `fraud_flags`; flags never block a purchase by themselves.
- **Modules:** auth, users, vendors, branches, staff, point-rules, rewards, cards, qr-codes, scans (collect + redeem), redemptions, fraud-flags, admin, reports.

5.4 QR code security NEW

The QR never contains the customer's id or anything personal: it holds a random one-time code (128 bits) issued by the server, e.g. `dvote:q1:Zx3...`.

Threat	Protection
Screenshot or photo of a QR reused later	Single use; expires after 5 minutes; a new QR cancels the previous one
Someone guesses another customer's QR	128-bit random code; nothing to guess from a user id
Database leak exposes live QR codes	Only the SHA-256 hash is stored
Staff of another vendor redeem a reward	Redeem QRs are tied to one vendor and reward (<code>vendor_mismatch</code>)
Same QR scanned on two staff phones at once	Conditional update: only the first succeeds
Points awarded twice for one receipt	One ledger row per QR; optional receipt number unique per branch

The app never decides points: it only shows the code. All API calls require a signed-in token over HTTPS.

5.5 Build order

1. Backend scaffold, Prisma, health check. DONE
2. Customer sign-in (Google, Facebook) and `/api/app/me` . DONE
3. Staff sign-in, invites and staff management. DONE
4. Vendor setup APIs: branches, point rule, rewards.
5. QR codes, collect flow and cards, with integration tests on a real PostgreSQL.
6. Redeem flow.
7. Reports, admin APIs, fraud-flag job, then the React dashboard.
8. React Native customer app and staff app (shared generated API client).

API Endpoints

Every call sends a Supabase access token. JSON is camelCase; errors carry a stable `code`.

6.1 Customer app (/api/app) — customer token

Method	Path	Purpose
GET / PATCH	/me BUILT	Profile; update name, email, phone (the profile is created on the first call)
GET	/cards	My cards: vendor, balance, lifetime points, next reward progress
GET	/cards/{id}/events	History of one card
GET	/vendors	Active vendors with branches and rule summary
GET	/vendors/{id}	Vendor page: branches, rule, rewards, my card (if any)
POST	/qr-codes	{purpose:"collect", vendorId?} or {purpose:"redeem", rewardId} → {id, code, expiresAt}
GET	/qr-codes/{id}	Status (active, used, expired, cancelled) and, once used, the result (+points or reward)
POST	/qr-codes/{id}/cancel	Customer closes the QR screen

6.2 Staff app (/api/vendor) — staff token **NEW**

Method	Path	Purpose
GET	/me BUILT	Signed-in staff member: role, vendor, branch
POST	/scans/preview	{code} → collect or redeem, the reward to serve, whether it can be used here. No customer personal data
POST	/scans/collect	{code, amount, receiptRef?, branchId?, idempotencyKey} → points added
POST	/scans/redeem	{code, branchId?, idempotencyKey} → reward approved
GET	/scans/today	The staff member's own collects and redemptions today

`branchId` is only sent by vendor admins (who have no branch); for everyone else the branch is their own.

6.3 Vendor dashboard (/api/vendor) — staff token, scoped to the staff member's vendor

Method	Path	Purpose
GET / POST	/staff BUILT	List staff; invite a new staff member by email
GET / PATCH	/staff/{id} BUILT	View; edit name, role, branch; disable

Method	Path	Purpose
GET / POST / PATCH	/branches	Own branches
GET / POST	/point-rules	Current rule and history; publish a new version (vendor admin)
GET / POST / PATCH	/rewards	Reward catalogue; archive instead of delete (vendor admin)
GET	/summary?from&to	Points issued, sales recorded, redemptions, unique and returning customers
GET	/events?branchId&from&to	Ledger rows for own branches
GET	/redemptions	Rewards given at own branches
GET	/fraud/flags	Flags for own vendor (read-only)

Vendors never see customer names or emails, only counts.

6.4 Admin dashboard (/api/admin) — platform admin token + 2FA

Method	Path	Purpose
GET / POST / PATCH	/vendors	Onboard and manage vendors, including their first vendor admin
GET	/vendors/{id}/summary	Any vendor's stats
GET / PATCH	/users/{id}	Customer lookup for support; block / unblock
POST	/cards/{id}/adjust	Manual point correction (adjust ledger row with reason)
GET / PATCH	/fraud/flags	Suspicious activity list; dismiss or confirm

6.5 Error codes for scans

Code	Meaning (shown to staff)
qr_invalid	Not a dvote QR, or unknown code
qr_used / qr_expired / qr_cancelled	Ask the customer to show a new QR
wrong_qr_type	A redeem QR was submitted as collect or the reverse
vendor_mismatch	The QR belongs to another vendor
branch_required	Vendor admin must choose a branch
no_active_rule	The vendor has not set a point rule yet
invalid_amount / no_points_earned	Amount not valid, or too small to earn a point
duplicate_receipt	This receipt number already earned points here

Code	Meaning (shown to staff)
reward_unavailable / insufficient_points	Reward archived, or the card no longer has enough points
user_blocked	Customer account blocked by the platform

Apps and Dashboards

What staff, vendors and platform admins can see and do.

7.1 Staff app NEW

- **Sign in** with the email and password from the invite. Vendor admins choose the branch they are working at.
- **Scan**: camera opens; scanning a customer's QR shows either the **Enter receipt total** form (collect) or **Redeem: Free coffee · 300 pts · Confirm** (redeem).
- **Result**: "+9 points added" or "Reward approved", or a clear error (Section 6.5).
- **Today**: the staff member's own collects and redemptions for the day.

7.2 Vendor vs admin dashboard

Feature	Vendor dashboard	Admin dashboard
Scope	One vendor and its branches	All vendors
Points issued, sales recorded per day / branch	Own only	All, comparable
Redemptions	Yes	Yes
Unique and returning customers	Counts only	Yes
Customer personal details	No	Yes, for support
Branches, staff	Yes	Yes
Point rule (e.g. 10 EGP = 1 pt)	Publish a new version	View, override
Reward catalogue	Add, edit, archive	View
Manual point correction	No	Yes, with reason
Fraud flags	Own, view only	All; dismiss or confirm
Vendor onboarding, billing	No	Yes

Tenant scoping. Every vendor-side query filters by the signed-in staff member's vendor (and branch for branch roles), enforced once in a guard. Rows of other vendors or branches are reported as not found.

Edge Cases and Fraud

What goes wrong in real cafés and how the system handles it.

8.1 Edge cases and failures

Situation	What happens	Handling
Points added, response lost on bad café wifi	Staff app retries	Same idempotency key → same result, no double points
QR expires while the customer waits in line	5 minutes pass	App shows a fresh QR automatically
Staff type a wrong total (e.g. 950 instead of 95)	Points already added	Platform admin corrects with an adjust row; <code>large_purchase</code> flag helps spot it. Later: vendor-side void within a few minutes
Receipt below one point (8 EGP)	0 points	<code>no_points_earned</code> ; nothing written; QR still usable
Customer shows an old screenshot	QR already used or expired	Rejected; customer opens a new QR
Vendor QR of Joy Corner scanned at Ecuador	Wrong vendor	<code>vendor_mismatch</code> ; the master QR works at any vendor
Two staff scan the same QR at once	Race	Conditional updates: exactly one succeeds
Customer redeems from two phones at once	Race on the balance	Conditional <code>UPDATE ... WHERE balance >= cost</code> : only one succeeds
Vendor changes its rule or a reward's price	Old points / redemptions	New rule applies from now on; past rows keep their rule version and reward snapshot
Reward archived while a redeem QR is open	Reward gone	<code>reward_unavailable</code> ; no points deducted
Staff member leaves	Account disabled	Blocked on the next request; past activity kept
Staff give points without a sale	Insider abuse	<code>staff_spike</code> , <code>too_many_collects</code> and <code>large_purchase</code> flags; vendor sees per-staff numbers
Customer has no internet at the counter	App cannot fetch a QR	v1: not supported (a QR needs the server). Later: pre-fetched codes

8.2 Automatic fraud flags

A scheduled job writes one `fraud_flags` row per finding, with the numbers in `details`. Flags never block anything on their own; a platform admin reviews them.

- `too_many_collects`: more than 3 collects for one customer in a day at one vendor.
- `large_purchase`: a receipt far above the branch's usual size (e.g. 5× its 30-day average).
- `branch_spike`: a branch's daily points above 3× its 30-day average.
- `staff_spike`: one staff member's daily points above 3× their own average.

Decisions and Roadmap

What is decided, and what comes later.

9.1 Decisions

- **Decided** **NEW**: spend-based points with a versioned rule per vendor; points round down.
- **Decided** **NEW**: each vendor defines a reward catalogue priced in points; the customer chooses when to redeem.
- **Decided** **NEW**: QR codes replace NFC. QRs hold a one-time server-issued code (5 minutes, single use), never the user id.
- **Decided** **NEW**: master QR (any vendor) and vendor QR (one vendor); the branch always comes from the scanning staff member.
- **Decided** **NEW**: a separate staff app (React Native) for scanning.
- **Decided**: Supabase Auth: Google and Facebook for customers, invited email + password for staff; Apple postponed.
- **Decided**: React Native + Expo for the mobile apps (replaces Flutter).
- **Removed**: NFC cards, NTAG 424 DNA, the 10-minute cooldown (one-time QRs and fraud flags replace it).

9.2 Later versions

Feature	Why	Needs
Push notifications	Confirm points even when the app is closed	FCM / APNs, device tokens table
Point expiry	Limits vendor liability for old points	Expiry rule on point_rules; ledger already has dates
Void a collect	Fix a mistyped total without the platform team	Vendor-side reversal within N minutes (adjust row)
Vendor promotions	Double points days, new-customer bonus	Promotion rules on top of point_rules
Offline QR	Customers without internet at the counter	Pre-fetched signed codes
POS integration	Receipt total sent automatically, no typing	Per-POS connectors
Apple sign-in	Required for an iOS release with social login	Apple Developer Program
Vendor billing	Monetise the platform	Plans table, invoices